

AI SUMMIT · Data Excellence · Tobacco Dock · London

Run | Keeping Models Reliable in Live Systems

“A model is a photograph of a moving world.”

THE PROBLEM

Production ML rarely fails because the model is wrong.

It fails because the world around the model changes.

Pipelines are rebuilt, upstream tables shift, feature definitions drift and the assumptions built into the model quietly stop matching reality. Accuracy erodes silently, long before anyone notices.

Broken decisions

Automated calls made on stale assumptions

Lost trust

Stakeholders stop believing the system

Operational surprise

Failures discovered by customers first

Three ideas in the next 25 minutes



1

What changes

Upstream schemas, data distributions, label timing and feature dependencies shift constantly usually without warning the model's owners.



2

How we detect it

Monitoring pipelines, drift and skew checks, slice metrics, explainability and human review make change visible early.



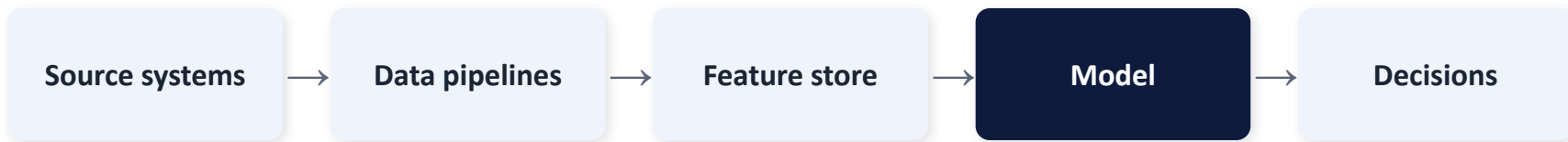
3

How we stay reliable

Data contracts, retraining policies, fallback logic and governance turn detection into a durable operating discipline.

Every model sits at the end of a dependency chain

The Model only consumes what the pipeline feeds it



Schema changes

Columns renamed, retyped or dropped without notice to consumers.



Freshness gaps

Late or missing partitions feed the model stale inputs.



Redefined features

“Active user” quietly means something new upstream.



Hidden dependencies

Features chain across teams nobody fully maps.

Three ways upstream change erodes performance



The dashboard says nothing changed. The world disagrees.



Data drift

Input distributions shift away from the training data
new users, new seasons, new behaviour.



Concept drift

The relationship between inputs and outcomes
changes with time in real world, the same signals now
mean something different.



Training-serving skew

Data used in production no longer matches the data
the model learned during training

When the world moves faster than the model

Grocery item-availability model, March 2020

93% →
61%

Reported prediction accuracy in a matter of weeks, as pandemic panic-buying rewrote shopping behaviour overnight.

What happened

Demand patterns shifted abruptly; historical training data no longer described the live world. The model was unchanged its assumptions were not.

The impact

Customers were promised items that were out of stock; downstream substitution and fulfilment decisions degraded with it.

What worked

Shortening the retraining window to use only the freshest data, and monitoring prediction quality continuously rather than at release.

A monitoring path that ends with a person



Alerts only matter if a named owner receives them. Teams that detect drift early pair automated checks with clear escalation, meaning detection without ownership is just a quieter way to fail.

Make change visible before customers feel it



Data quality checks

Validate schema, null rates, volume and freshness on every batch, catch breakage at the door, not in the metrics.



Statistical drift tests

Compare live feature and prediction distributions against training baselines (e.g. PSI, KS tests) on a schedule.



Slice metrics

Track performance per segment and region. Global averages hide the niches where models fail first.



Explainability signals

Shifts in feature attributions reveal which input changed, turning 'accuracy dropped' into 'this dependency moved'.

Detection is only half the job, Is your team ready to tackle drift?



Retraining triggers

Retrain when drift signals fire, not on the calendar and validate every refresh before it ships.



Deployment gating

Canary and shadow deployments compare the new model against live traffic before full rollout.



Fallback paths

A tested plan B last-known-good model or simple rules, keeps decisions flowing when quality drops.



Escalation playbooks

Human-in-the-loop review for high-stakes calls, with named owners and rehearsed runbooks.

Automatic where possible, human-supervised where necessary.

A dashboard is not a response

Monitoring without ownership

- Dashboards exist, but nobody is paged
- Alerts fire with no runbook attached
- Retraining ships without validation gates
- No time or budget reserved for incidents

Monitoring with ownership

- Every model has a named, accountable owner
- A reliability budget funds response, not just builds
- Playbooks are rehearsed before they are needed
- Postmortems feed back into data contracts

Frameworks for reliability that lasts



Socio-technical lens

Reliability lives in teams and processes, not just code. Design the handoffs between data producers, ML engineers and operators as deliberately as the architecture.



Contracts & lineage

Data contracts make upstream expectations explicit and breaking changes negotiable; lineage makes every dependency traceable when something moves.



Risk frameworks

Approaches like the NIST AI Risk Management Framework govern, map, measure, manage match the rigour of controls to the impact of the decision.

THE TAKEAWAY

The reliability checklist

- ✓ Every upstream dependency has a contract
- ✓ Metrics are sliced, not just averaged
- ✓ A fallback path exists and has been tested
- ✓ Drift and skew are monitored, not assumed
- ✓ Retraining has triggers and validation gates
- ✓ Every alert reaches a named owner

Make upstream change visible, Downstream impact measurable.

Make response automatic where possible. Human-supervised where necessary.